SKILLNAVORA · AI AND ML · TECH FOUNDATIONS FOR PRODUCT MANAGERS

# System Design & Databases for PMs

From the whiteboard to the business: how an app like Zepto sells the last item safely. SQL, NoSQL, Cassandra, ACID, CAP, Paxos, vector databases, geolocation, queues and interview practice, in plain words.

Edition 2026 · skillnavora.in

# How to use this guide

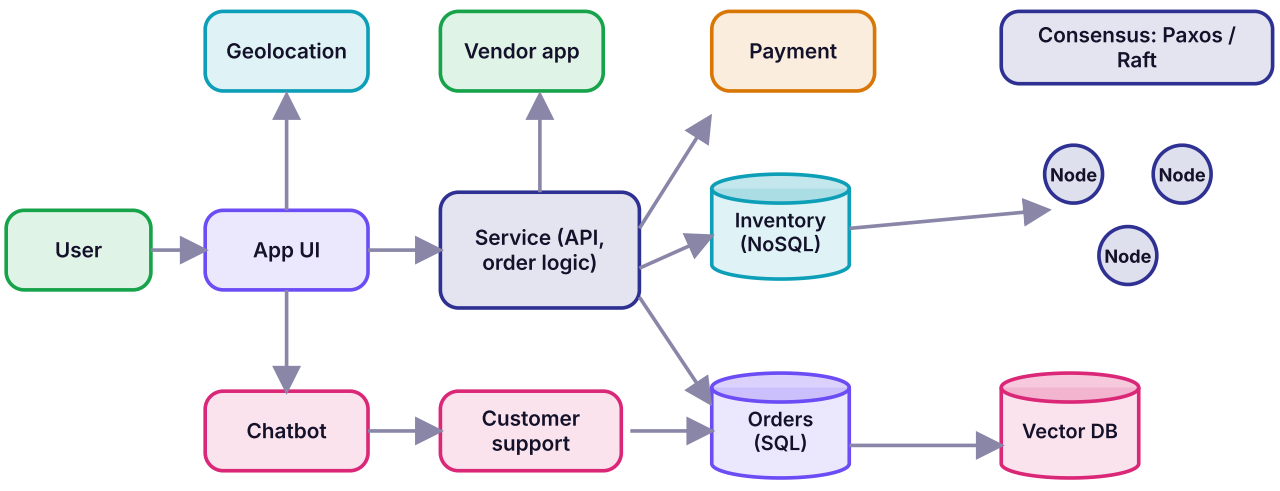
Each class has goals, concepts with Indian examples, key takeaways and a practice exercise, from basic to advanced. Finish with the final project to build something you can show in interviews.

| | Class | Level |
|---|---|---|
| Class 1 | **The big picture: what happens when you tap Buy** | Basic |
| Class 2 | **Why PMs need this: the business view** | Basic |
| Class 3 | **SQL databases in plain words** | Basic |
| Class 4 | **ACID: why money and stock stay correct** | Basic |
| Class 5 | **NoSQL: the main types and when to use them** | Intermediate |
| Class 6 | **Cassandra explained for PMs** | Intermediate |
| Class 7 | **Scaling data: replication, sharding, caching, CAP and BASE** | Intermediate |
| Class 8 | **Services and servers: what happens when volume grows** | Intermediate |
| Class 9 | **The inventory problem: never sell the last item twice** | Intermediate |
| Class 10 | **The full user journey: every step, system and risk** | Intermediate |
| Class 11 | **Payment checkout in detail** | Advanced |
| Class 12 | **Never twice: no repeated order, payment or customer** | Advanced |
| Class 13 | **Consensus: Paxos and Raft without the maths** | Advanced |
| Class 14 | **Vector databases, geolocation and the AI parts** | Advanced |
| Class 15 | **The supporting cast: APIs, queues, events and observability** | Advanced |
| Class 16 | **Case study and PM interview answer: design inventory and checkout** | Advanced |
| Class 17 | **Final project: write a one-page system brief** | Advanced |
| | **Toolkit:** templates, key terms cheat sheet, glossary, 20 interview questions, further reading | |

BASIC · CLASS 1

# The big picture: what happens when you tap Buy

See every box on the whiteboard in plain words.




The whiteboard as a system: what talks to what when a customer taps Buy

Imagine a quick-commerce app like Zepto or Blinkit. You open it, see milk, tap **Buy**. In about two seconds, a dozen systems cooperate. Here is each box, explained for a non-technical PM.

| Box | Plain-words job | Everyday comparison |
|---|---|---|
| **User / App UI** | The screen you tap. It asks the service for data and shows the answer. | The shop counter |
| **Geolocation** | Finds where you are and which nearby store can deliver. | Google Maps on the counter |
| **Service (API)** | The brain with the business rules: check stock, price, create the order. Many small services is called *microservices*. | The shop manager |
| **Inventory database** | Stores how many items each store has. Read thousands of times a second. | The stock register |
| **Orders database (SQL)** | Stores orders, payments, refunds exactly and safely. | The cash book |
| **Payment** | Talks to UPI, cards and wallets through a gateway such as Razorpay. | The card machine |
| **Vendor app** | The store or seller updates stock and prices and accepts orders. | The shopkeeper's phone |
| **Chatbot + customer support** | Answers "where is my order?" and hands tough cases to humans. | The help desk |
| **Vector database** | Lets AI search by meaning, so the chatbot finds the right help article. | A librarian who understands what you mean |
| **Consensus nodes (Paxos / Raft)** | A few servers that must agree on one truth, even if one fails. | Four partners who must agree on the same ledger |

**Your job as a PM** is not to draw this perfectly. It is to ask the right "what happens when..." questions: when two people buy the last item, when payment fails after stock is taken, when a server dies on sale day.

**Key takeaways**

- Every tap triggers many systems working together.
- Each box has one job; failures happen between boxes.
- You do not build it, but you must be able to question it.

**Practice exercise:** Draw your favourite app's Buy flow with 8 boxes and write one 'what if this fails?' question for each arrow.

BASIC · CLASS 2

# Why PMs need this: the business view

Connect technical choices to money, trust and speed.

System design looks technical, but every choice is a business trade-off. Engineers decide *how*; PMs decide *what matters most*. Without that, teams over-build or under-protect.

| Business goal | Technical lever | PM decision |
|---|---|---|
| More sales | Fast pages and search (caching, indexes, CDN) | What is our target load time on a 4G phone? |
| Customer trust | No double charge, no oversold stock (transactions, idempotency) | Which actions must never be wrong, even if slower? |
| Sale-day survival | Scaling, queues, replicas, load tests | What traffic do we plan for, 5x or 20x? |
| Lower cost per order | Right database for each job, storage tiers | What can we afford per 1,000 orders? |
| Legal safety | Encryption, data location, access control (DPDP Act, RBI rules, PCI-DSS) | Which data is sensitive, and who may see it? |
| Fast shipping of features | Clear services and APIs | Which team owns which part? |

## Numbers PMs should know

- **Latency:** time for one request. Watch **p95/p99** (the slow 5% or 1% of users), not only the average.
- **Availability:** 99.9% allows about 8.8 hours of downtime a year; 99.99% only about 53 minutes.
- **SLA / SLO / SLI:** the promise to customers / the internal target / the measured number.
- **RPO / RTO:** how much data you can lose / how long you can be down after a disaster.

A cheap rule: **spend reliability budget where mistakes cost real money or trust** (payments, stock, personal data) and accept simpler, cheaper setups where a small error is harmless (likes, view counts).

**Key takeaways**

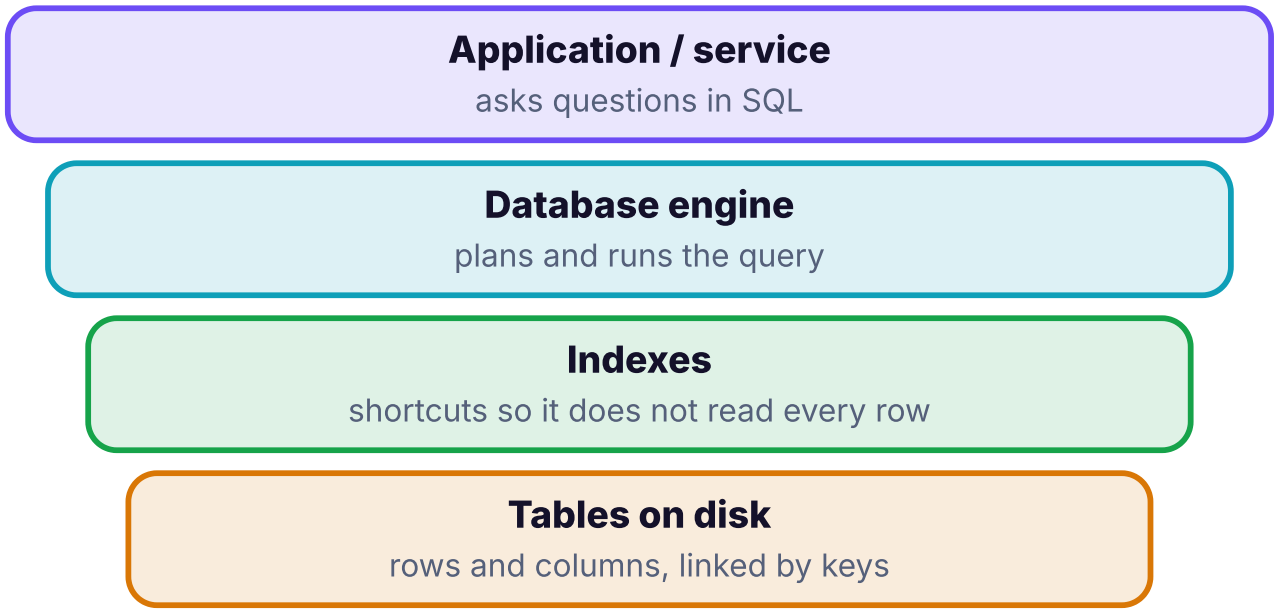
- Technical choices are business trade-offs.
- Protect money, stock and personal data first.
- Ask for p95/p99 and availability targets in every spec.

**Practice exercise:** Pick a feature you own. List the one thing that must never be wrong and the one thing that can be a little stale.

BASIC · CLASS 3

# SQL databases in plain words

Understand tables, keys, joins and indexes without code fear.




How a SQL database answers a question

A SQL (relational) database is a set of tables, like linked Excel sheets. Each row is a record; **keys** link rows across tables. SQL is the language to ask questions. Examples: PostgreSQL, MySQL, SQL Server, Oracle.

| Table | Columns |
|---|---|
| products | product_id (key), name, price |
| inventory | store_id, product_id, qty, state |
| orders | order_id (key), user_id, store_id, total, status |

```
-- How many packets of milk are available in store 7?
SELECT qty FROM inventory
WHERE store_id = 7 AND product_id = 42 AND state = 'available';
```

- **Primary key:** the unique ID of a row. **Foreign key:** a column pointing to another table's key.
- **Join:** combine tables ("orders with the customer's name").
- **Index:** a lookup shortcut, like a book's index. Makes reads fast but writes a little slower and uses space.
- **Schema:** the fixed structure of tables. Changing it needs a planned migration.

> **Choose SQL when** data is structured, relationships matter, and correctness is critical: orders, payments, accounts, bookings, reports.

**Key takeaways**

- SQL = tables with keys, queried by joins.
- Indexes trade write speed for read speed.
- Default to SQL for money and orders.

**Practice exercise:** Sketch three tables for a food-delivery app and mark the primary and foreign keys.

BASIC · CLASS 4

# ACID: why money and stock stay correct

Learn the four promises of a database transaction.

A **transaction** groups steps so they succeed or fail together. Example: buying the last milk packet means (1) reduce stock, (2) create the order, (3) record the payment. If step 3 fails, steps 1 and 2 must undo.

| Letter | Promise | Plain example |
|---|---|---|
| **A**tomicity | All steps happen or none do | Money leaves your account only if it reaches the shop |
| **C**onsistency | Rules always hold | Stock never goes below zero |
| **I**solation | Simultaneous users do not corrupt each other | Two buyers cannot both get the last item |
| **D**urability | Once confirmed, it survives a crash | Your order still exists after a power cut |

```
BEGIN;
UPDATE inventory SET qty = qty - 1
  WHERE store_id = 7 AND product_id = 42 AND qty > 0;
INSERT INTO orders (user_id, store_id, status) VALUES (101, 7, 'created');
COMMIT;   -- or ROLLBACK if anything failed
```

Transactions are why SQL databases are trusted for payments. Many NoSQL databases trade some of these promises for speed and scale, which is the key trade-off in the next classes.

**Key takeaways**

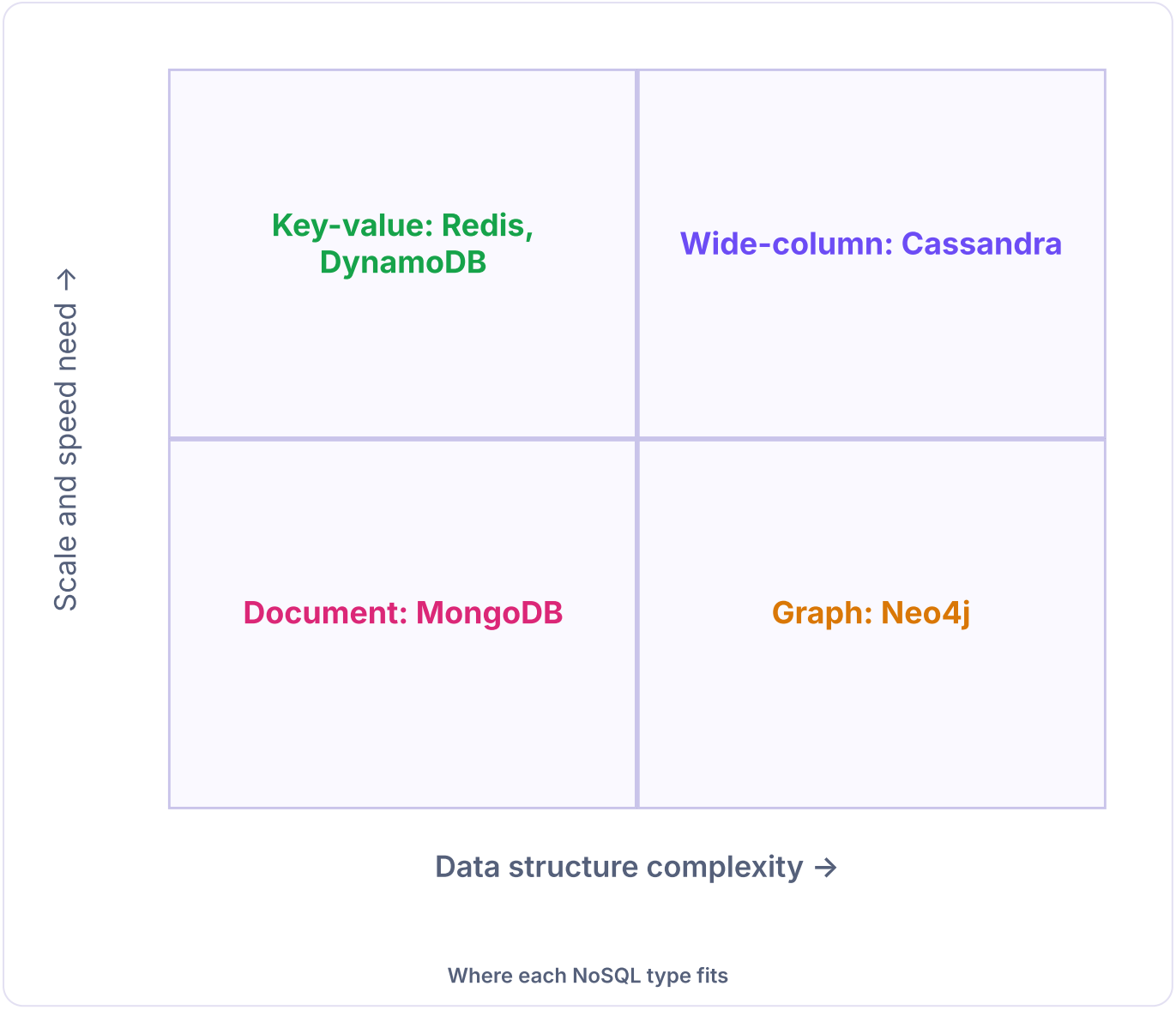
- Transaction = all-or-nothing group of steps.
- ACID protects money and stock.
- Isolation is the promise behind 'no double selling'.

**Practice exercise:** Write the steps of a UPI payment and mark which must be all-or-nothing.

INTERMEDIATE · CLASS 5

# NoSQL: the main types and when to use them

Pick between key-value, document, wide-column and graph stores.




Where each NoSQL type fits

**NoSQL** means "not only SQL": databases that skip fixed tables and joins to scale across many servers and handle flexible data. They usually relax some ACID promises for speed (see BASE in class 7).

| Type | Shape | Good for | Examples |
|---|---|---|---|
| Key-value | One key → one value | Sessions, carts, caches, counters, feature flags | Redis, DynamoDB |
| Document | JSON-like documents | Product catalogue, user profiles with varied fields | MongoDB, Firestore |
| Wide-column | Rows grouped by a partition key, huge and sparse | Messages, activity feeds, time-series, IoT, write-heavy data | Cassandra, ScyllaDB, HBase |
| Graph | Nodes and relationships | Friends, fraud rings, recommendations | Neo4j, Neptune |
| Vector | Numbers that capture meaning | AI search and chatbots (class 14) | Pinecone, pgvector, Qdrant |

## SQL vs NoSQL in one table

| | SQL | NoSQL |
|---|---|---|
| Structure | Fixed tables | Flexible |
| Relationships | Joins are easy | Usually avoided or hand-built |
| Consistency | Strong (ACID) | Often eventual, tunable |
| Scaling | Vertical first, harder to spread | Built to spread across many servers |
| Best for | Orders, payments, reports | High-volume reads/writes, flexible data |

**Common real-world answer:** use both. SQL for orders and money, NoSQL (Redis/Cassandra/Mongo) for carts, catalogue, feeds and caches. This is called *polyglot persistence*.

**Key takeaways**

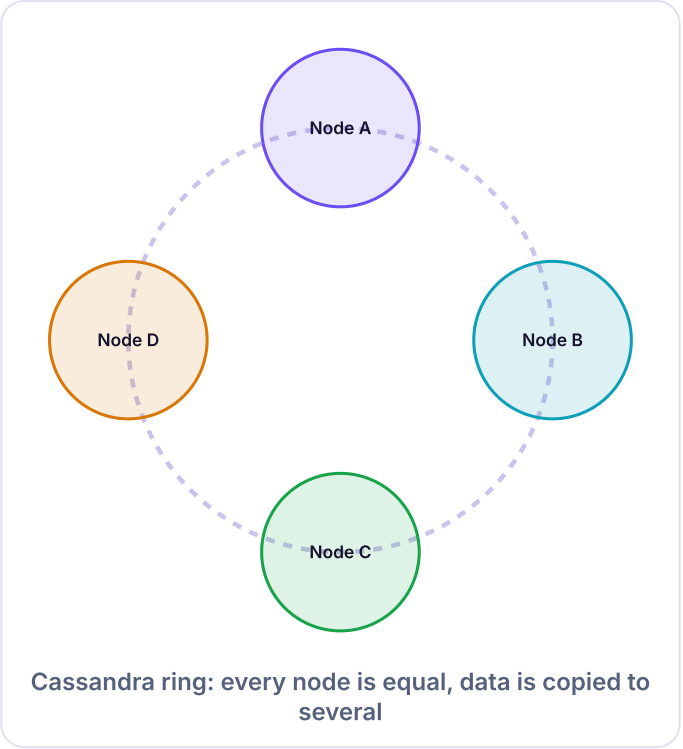
- NoSQL has four main shapes plus vector.
- It trades some consistency for scale and flexibility.
- Most products use SQL and NoSQL together.

**Practice exercise:** Match five data types in a food app (cart, orders, menu, chat, ratings) to a database type.

INTERMEDIATE · CLASS 6

# Cassandra explained for PMs

Understand a masterless, always-on database.




Cassandra ring: every node is equal, data is copied to several

**Apache Cassandra** is a wide-column NoSQL database built so there is **no single point of failure**. All servers (nodes) are equal and form a ring. It is popular where data arrives constantly and downtime is unacceptable: messages, activity feeds, order status events, sensor data.

- **Partition key:** the field that decides which node stores a row (for example `store_id`). Chosen badly, one node gets overloaded (a "hot partition").
- **Replication factor (RF):** how many copies exist. RF=3 means each row lives on three nodes.
- **Tunable consistency:** per query you choose how many copies must answer. **ONE** = fastest, maybe stale; **QUORUM** = majority, safer; **ALL** = strongest, slowest.
- **Fast writes:** it appends data first and tidies later (an LSM-tree design).
- **Query-first design:** you design tables around the questions you will ask. Ad hoc queries and joins are weak.

| Strengths | Weaknesses |
|---|---|
| Writes at massive scale, multi-region, survives node loss | No joins, limited ad hoc queries, transactions are limited and slow |
| Linear scale: add nodes to add capacity | Needs careful data modelling and operations skills |

## LWT (lightweight transactions) and the other terms you will hear

**LWT** is Cassandra's way to do a safe "check, then write" in one step, for example `INSERT … IF NOT EXISTS` (claim a username, create an order once) or `UPDATE … IF qty > 0`. Under the hood the replicas run a **Paxos** round (class 14) to agree, so it needs several network round trips: about 4

instead of 1. It is correct but **much slower** and gets worse when many people fight for the same row. Use it sparingly (claiming a unique ID, one-time actions), never on the hot path of every click.

```
INSERT INTO orders_by_key (idempotency_key, order_id, user_id)
VALUES ('chk-7f3a', 55, 101) IF NOT EXISTS;   -- LWT: only the first request wins
```

| Term | Plain meaning |
|---|---|
| **CQL** | Cassandra's SQL-like query language |
| **Partition key / clustering key** | Partition key decides *which node*; clustering key decides *the order of rows inside* that partition (for example newest first) |
| **Consistency levels** | ONE (fast), QUORUM (majority), LOCAL_QUORUM (majority inside one data centre, the usual choice for multi-region), ALL (every copy), SERIAL (used by LWT) |
| **Gossip** | Nodes chat with each other to learn who is up or down; no central boss |
| **Commit log + memtable + SSTable** | A write goes to a safety log and to memory first, then is flushed to immutable files on disk (SSTables). This is why writes are fast |
| **Compaction** | Background merging of SSTables to keep reads fast and reclaim space |
| **Tombstone** | A delete marker. Many tombstones slow reads, so avoid patterns that delete constantly (queues in a table) |
| **TTL** | Time to live: rows that expire automatically (OTPs, sessions, 8-minute stock holds) |
| **Hinted handoff, read repair, anti-entropy repair** | The ways copies that missed a write catch up, so replicas converge (eventual consistency) |
| **Replication strategy / data centre** | NetworkTopologyStrategy places copies across racks and regions; this is how Cassandra survives a whole data centre going down |
| **Vnodes** | Each server owns many small slices of the ring so data spreads evenly and new nodes take load smoothly |
| **Denormalisation** | Store the same data in several tables, one per query, because there are no joins |

**PM question:** "Do we need always-on writes at huge scale, or do we need strict transactions?" If the answer is strict transactions (payments, stock decrement), choose SQL or a database with strong transactions instead.

**Key takeaways**

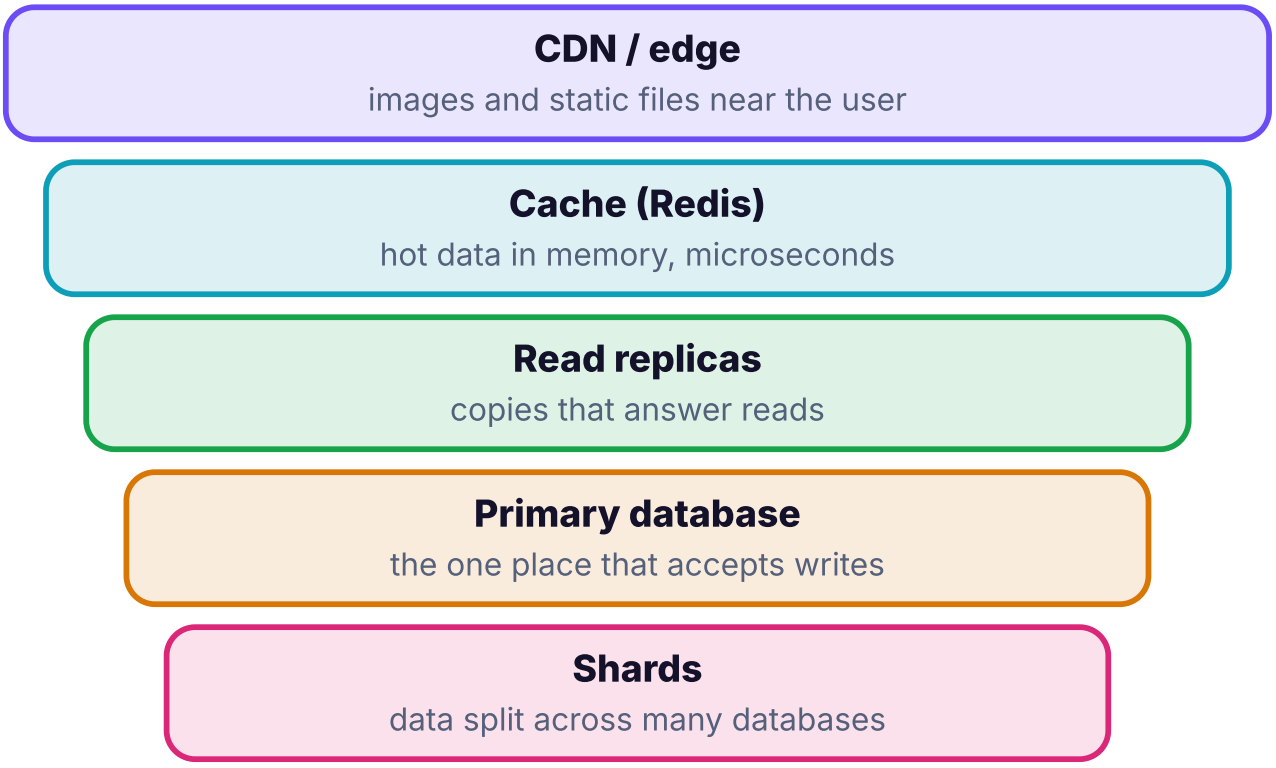
- Cassandra = equal nodes in a ring, data copied several times.
- You choose consistency per query.
- Great for high-volume writes, poor for joins and strict transactions.
- LWT adds safe check-then-write using Paxos, but is slow: use it sparingly.

**Practice exercise:** List three features in a delivery app that fit Cassandra (hint: tracking pings, chat, event logs) and one that does not.

INTERMEDIATE · CLASS 7

# Scaling data: replication, sharding, caching, CAP and BASE

Know how products stay fast and available as they grow.




From the user to the data: layers that make products fast

| Technique | Plain meaning | Trade-off |
|---|---|---|
| **Replication** | Keep copies on several servers for safety and read speed | Copies can lag (stale reads) |
| **Sharding / partitioning** | Split data by a key (city, store, user id range) across servers | Cross-shard queries and transactions are harder |
| **Caching** | Keep popular answers in memory | Must expire or refresh, or users see old data |
| **Indexing** | Pre-built lookups | More storage, slower writes |
| **Load balancer** | Spreads requests across many servers | Needs health checks |
| **CDN** | Serves images and files from nearby servers | Cache invalidation |

## CAP theorem in one minute

When a network splits and servers cannot talk to each other, a system must choose between **Consistency** (everyone sees the same truth, so some requests are refused) and **Availability** (always answer, maybe with stale data). **Partition tolerance** is not optional on a real network.

| Choose consistency (CP) | Choose availability (AP) |
|---|---|
| Payments, final stock decrement, seat booking | Product browsing, recommendations, view counts, "30 people watching" |

**BASE** is the NoSQL counterpart of ACID: *Basically Available, Soft state, Eventually consistent*. "Eventually" means copies agree after a short delay, often milliseconds.

> **Quorum rule:** with N copies, if reads (R) and writes (W) each touch enough copies so that **R + W > N**, a read always sees the latest write. Example: N=3, R=2, W=2.

**Key takeaways**

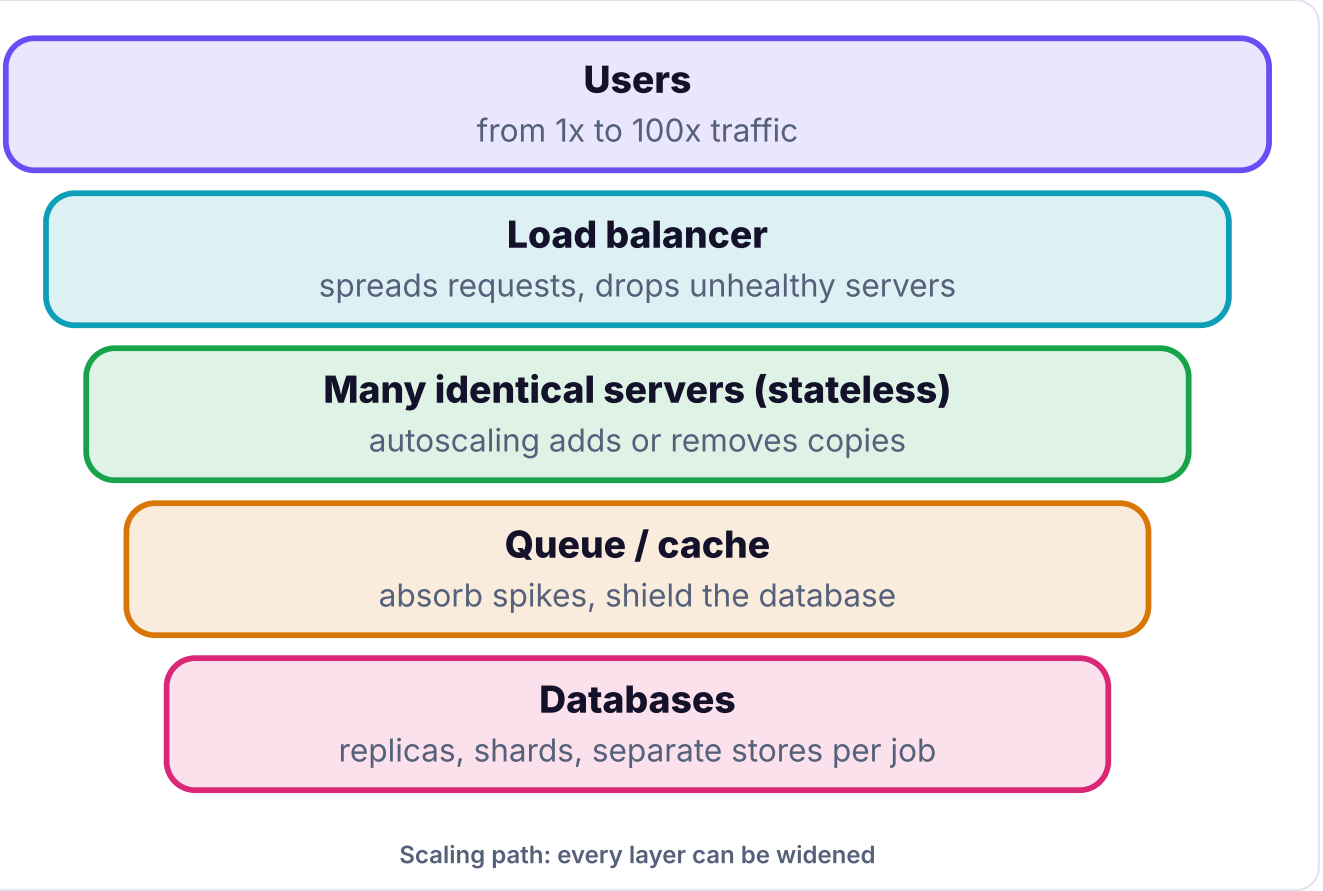
- Replication, sharding, caching and indexing are the four scaling tools.
- CAP: during a network split you cannot have both perfect consistency and perfect availability.
- Use strong consistency only where wrong data costs money or trust.

**Practice exercise:** Classify ten features of a shopping app as 'must be exact' or 'can be a bit stale'.

INTERMEDIATE · CLASS 8

# Services and servers: what happens when volume grows

Follow the app from 100 users to 10 million without the jargon.




Scaling path: every layer can be widened

A **server** is a computer running your code. A **service** is one job done by that code (catalogue, orders, payments). As more people use the app, you do not need smarter code first; you need **more copies of the same code** and a database layer that can keep up.

| Traffic | What the team does | PM impact |
|---|---|---|
| **1x: launch day** | One server and one database are enough. Keep it simple. | Ship features fast |
| **5x** | Add a **load balancer** and several identical servers; add a **cache** for hot data; add a **read replica**. | Page speed stays steady |
| **20x: a sale or festival** | **Autoscaling** adds servers on demand; **queues** take slow work (emails, invoices) off the checkout path; **rate limits** protect the system. | Plan promotions with engineering; pre-warm for known peaks |
| **100x: a national app** | **Shard** the database by city or user; split into **microservices**; use several regions; add a **CDN** for images. | Cost and team ownership become the main questions |

## The key ideas

- **Vertical scaling** = a bigger machine (simple, has a ceiling). **Horizontal scaling** = more machines (no ceiling, needs the design below).

- **Stateless services:** a server keeps no customer data in its own memory, so any server can answer any request and one can fail without anyone noticing. State lives in the database, cache or session store.
- **Autoscaling:** rules like "add a server when CPU is above 60%". It takes a minute or two, so it cannot catch a sudden 50x spike; prepare ahead.
- **Containers and Kubernetes:** packaging code so identical copies start fast and are managed automatically.
- **Queues and backpressure:** when work arrives faster than it can be done, put it in line instead of crashing; slow down or reject extra requests politely (HTTP 429).
- **Graceful degradation:** when overloaded, switch off the non-essential (recommendations, live counters) to protect checkout.
- **Bottlenecks move:** after servers, the database is next; after the database, a third-party API such as the payment gateway. Find the weakest link with a **load test**.
- **Circuit breakers and load shedding:** stop calling a failing dependency so one problem does not become a full outage.

**Sale-day checklist for a PM:** expected peak orders per second, load test passed at 2x that, autoscaling limits raised, cache warmed, payment gateway limits confirmed in writing, feature flags ready to switch off extras, on-call named, rollback plan.

**Key takeaways**

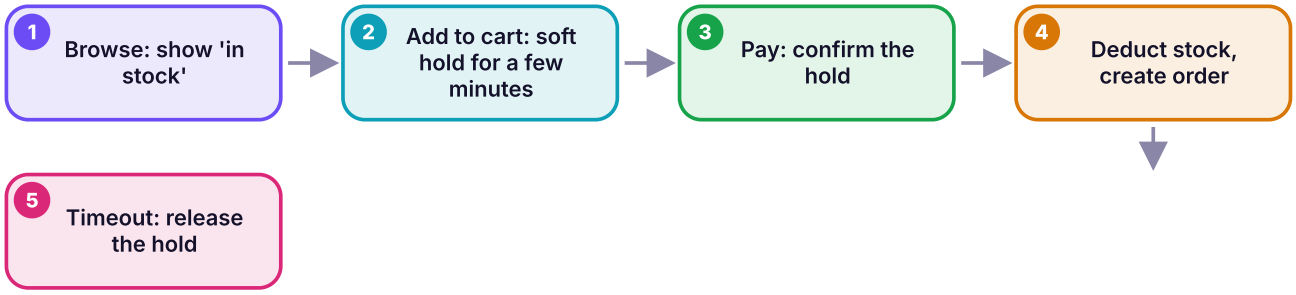
- Scale by adding identical stateless servers behind a load balancer.
- Protect the database with caches, replicas and queues.
- Find the next bottleneck with load tests before the sale, not during it.

**Practice exercise:** Write a sale-day plan for a 20x traffic spike: what scales, what is switched off, and who is on call.

INTERMEDIATE · CLASS 9

# The inventory problem: never sell the last item twice

Walk the whiteboard's qty = 1 problem and its fixes.




Inventory reservation flow

Two customers see "1 left" and tap Buy at the same moment. If both read `qty = 1` and both subtract, you sell two and have one. This is a **race condition**. The fix is to make the check and the subtraction one indivisible step.

| Approach | How it works | Good for |
|---|---|---|
| **Atomic update** | `UPDATE … SET qty = qty - 1 WHERE qty > 0`; if 0 rows changed, it is sold out | Simple, most cases |
| **Pessimistic lock** | Lock the row while one buyer decides | Rare, very hot items; can slow everyone |
| **Optimistic lock** | Read a version number; update only if it has not changed; else retry | Many readers, fewer conflicting buyers |
| **Reservation (hold)** | Reserve the item for 5-10 minutes during checkout, then confirm or release | Tickets, tatkal seats, flash sales |
| **Queue per item** | Put buyers in line and process one by one | Extreme flash sales |

## Other rules that protect money

- **Idempotency key:** a unique request ID so a retried tap or a repeated payment callback does not create a second order or second charge.
- **Two-phase commit (2PC):** several systems agree to commit together; strong but slow and fragile.
- **Saga:** a chain of steps, each with an undo ("refund", "release stock"). Popular in microservices.
- **Outbox pattern:** write the order and the "order created" event together, then publish the event, so none gets lost.

**Product choices:** how long is a hold (stock locked vs lost sales)? Do we show "only 2 left" (real-time cost vs urgency)? What do we do when payment fails after stock is held (auto-release, notify)?

**Key takeaways**

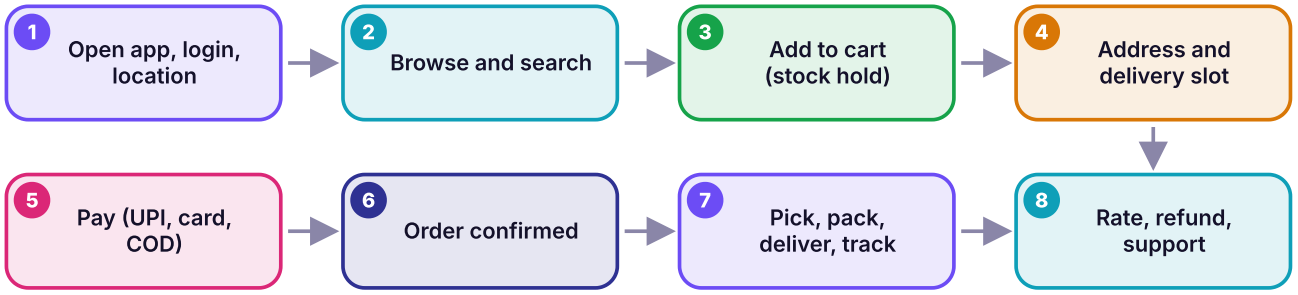
- The last-item problem is a race; fix it with one atomic step.
- Holds with a timeout balance sales and fairness.
- Idempotency keys stop double orders and double charges.

**Practice exercise:** Define hold time, release rule and message for a 'last seat' booking feature.

INTERMEDIATE · CLASS 10

# The full user journey: every step, system and risk

Follow one order from opening the app to delivery and refunds.




The customer journey as a chain of systems

| # | What the user does | Systems involved | Data | What can go wrong, and the protection |
|---|---|---|---|---|
| 1 | Opens the app, logs in with OTP, shares location | Auth service, SMS provider, geolocation | User profile (SQL), session (Redis) | OTP abuse (rate limit); duplicate accounts (unique verified phone); location consent |
| 2 | Sees the nearest store and products | Geo service, catalogue, search, vector search, cache, CDN | Catalogue (document store), stock snapshot (cache) | Stale stock (label as approximate); slow load (cache, CDN) |
| 3 | Adds items to the cart | Cart service, inventory service | Cart (key-value store), soft hold on stock | Cart lost (persist per user); hold never released (TTL) |
| 4 | Chooses address and slot, sees the total | Address, pricing, offers, tax services | Addresses and coupons (SQL) | Price changes mid-checkout (price lock per cart version); coupon abused (one use per user) |
| 5 | Taps Pay | Order service, payment gateway (UPI, cards, wallets) | Order (SQL) in status PENDING, payment record | Double tap or retry (idempotency); money taken but no order (reconciliation) |
| 6 | Sees "Order confirmed" | Webhook handler, order and inventory services, notifications | Order PAID, stock deducted | Late or repeated webhooks (dedupe by payment id); stock gone (auto refund) |
| 7 | Tracks the order | Vendor app, picker app, rider app, maps, event stream | Status events and rider pings (wide-column / stream) | Missed updates (event replay); wrong ETA (re-estimate) |
| 8 | Delivered; rates, returns or asks for help | Support, chatbot with vector search, refund service | Tickets and refunds (SQL), help articles (vector DB) | Refund repeated (refund idempotency); bot gives a wrong answer (guardrails, human handover) |

## Journey metrics a PM watches

- **Funnel:** browse, cart, checkout started, payment attempted, payment succeeded, delivered. Each drop has a cause: speed, trust, price, payment failure.
- **Payment success rate** (by method and bank), **order success rate**, **p95 checkout time**, **cancellation and refund rate**, **duplicate order rate** (target zero).
- **Support contacts per 100 orders** as a quality signal for the whole chain.

**Order states** (a simple state machine): CREATED, PAYMENT_PENDING, PAID, PACKED, OUT_FOR_DELIVERY, DELIVERED, with side exits CANCELLED, FAILED and REFUNDED. Only allowed moves are accepted, which is what makes repeated or out-of-order messages harmless.

**Key takeaways**

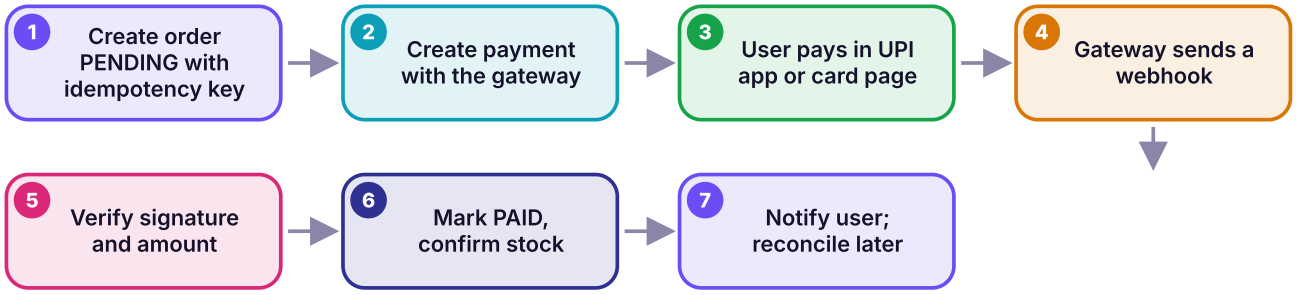
- Every journey step touches different systems and data stores.
- Each step has a named risk and a named protection.
- An order state machine keeps messy real life under control.

**Practice exercise:** Map your own app's journey in the same table and add the metric you would track at each step.

ADVANCED · CLASS 11

# Payment checkout in detail

Understand how a payment really moves, and why it is asynchronous.




Checkout and payment: the safe sequence

Payments are **asynchronous**: your app starts a payment, the user finishes it in another app (UPI, bank page), and the gateway tells you the result later. The screen result can be wrong or lost, so the **server-to-server confirmation** is the truth.

1. **Create the order as PENDING** in SQL with a unique **idempotency key** made when the user starts checkout. Price, items and address are frozen on the order.
2. **Create a payment** with the gateway (Razorpay, PayU, Cashfree, Stripe, UPI intent). You get a **payment id**. Store it against the order.
3. **The user pays** in the UPI app or card page. Your app only shows "waiting for payment".
4. **The gateway calls your webhook** with the result. Verify the **signature** and that the **amount and order id match**. Never trust the client app's "success".
5. **Update in one transaction:** mark the order PAID, convert the stock hold into a deduction, write a ledger entry.
6. **Publish an event** (outbox) so notifications, warehouse and analytics follow.
7. **Reconcile:** a scheduled job compares your records with the gateway's settlement report and fixes mismatches (paid but no order, order but no payment).

| Payment state | Meaning | What the app does |
|---|---|---|
| CREATED | Payment started | Wait; show a countdown |
| PENDING | Bank or UPI has not confirmed | Poll the gateway; do not charge again |
| AUTHORISED / CAPTURED | Money held / money taken | Confirm the order after capture |
| FAILED | Declined or timed out | Release stock, offer retry |
| REFUNDED / PARTIAL | Money returned | Update the order; notify |

## Edge cases to ask about

- **Money debited but the app shows failure:** keep the order PENDING until the gateway confirms; auto-refund after a defined window if the order cannot be fulfilled.
- **Webhook arrives twice, late or out of order:** handlers must be idempotent and obey the state machine.

- **User closes the app mid-payment:** the server still receives the webhook, so the order completes.
- **Gateway down:** fail over to a second gateway or show clear messaging; do not lose the cart.
- **Cash on delivery:** no gateway step, but the order still needs duplicate protection and fraud limits.
- **Security:** never store raw card numbers (use gateway tokens, PCI-DSS); encrypt personal data; follow RBI rules and the DPDP Act.

**Key takeaways**

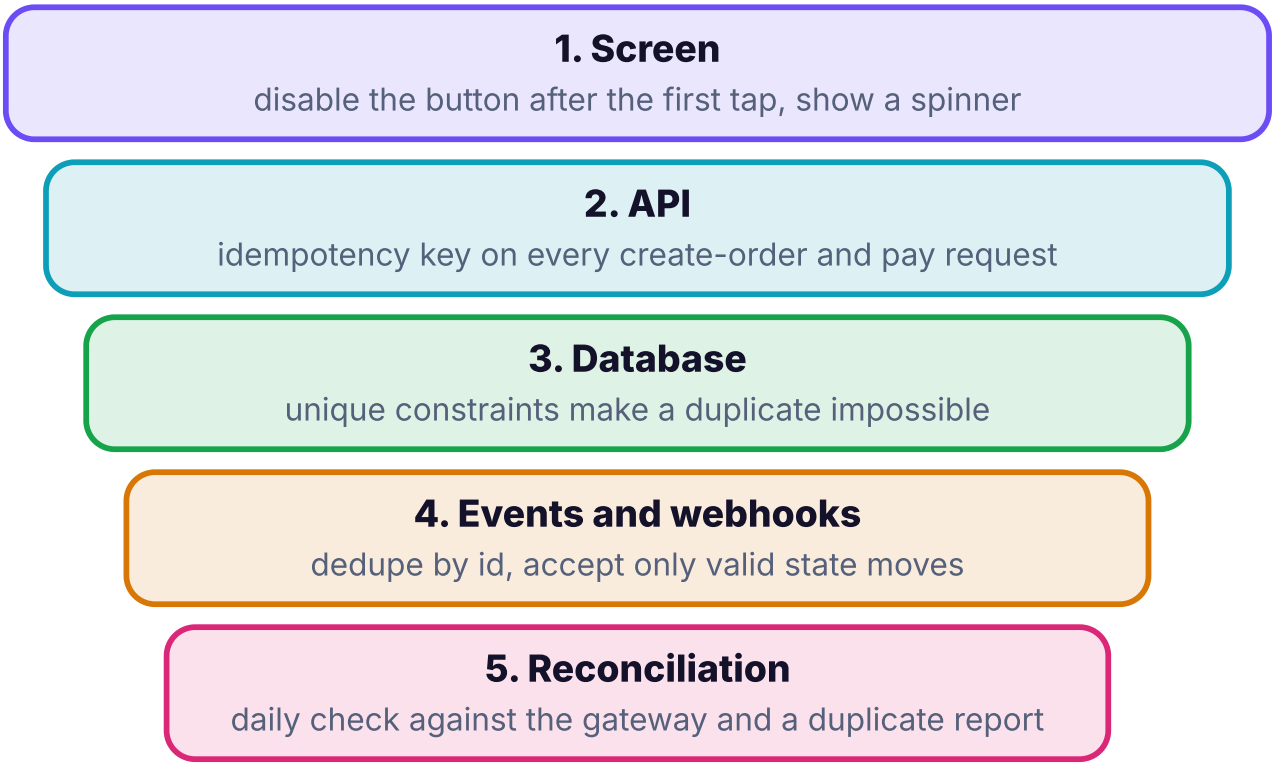
- The server-side webhook, not the app screen, decides success.
- Create the order first as PENDING, then confirm after verified payment.
- Reconciliation catches whatever the real-time flow misses.

**Practice exercise:** Draw the sequence of a UPI payment with five actors: user, app, your server, gateway, bank. Mark where a failure can occur.

ADVANCED · CLASS 12

# Never twice: no repeated order, payment or customer

Use layered protections so duplicates cannot happen, even with retries.




Five layers against duplicates

Networks fail, users double tap, apps retry, and webhooks repeat. Systems normally guarantee **at-least-once** delivery (a message may arrive twice). True "exactly once" is built by combining **at-least-once delivery** with **idempotent processing**. You need protections at several layers because any single one can fail.

## A. How to make sure an order is not repeated

- **Idempotency key:** the app generates one key per checkout attempt (for example cart id plus cart version). Every retry sends the same key. The server stores key to order id. A second request with the same key returns the **same order** instead of making a new one.
- **Unique constraint in SQL:** a unique index on `(user_id, idempotency_key)` means the database itself refuses a second row, even if the code has a bug or two servers race.
- **One active checkout per cart:** a short lock (Redis `SET key value NX EX 30`, or a row lock) so two simultaneous taps on two servers cannot both proceed.
- **Order state machine:** only legal moves are accepted, so a repeated "mark as paid" does nothing.
- **Time window rule:** if the same user, store and basket is submitted twice within a few seconds, show "you already placed this order" with a link to it.

```
CREATE UNIQUE INDEX uq_order_idem ON orders (user_id, idempotency_key);

INSERT INTO orders (user_id, idempotency_key, status, total)
VALUES (101, 'chk-7f3a', 'PAYMENT_PENDING', 349)
ON CONFLICT (user_id, idempotency_key) DO NOTHING;   -- a repeat inserts nothing
SELECT order_id FROM orders WHERE user_id = 101 AND idempotency_key = 'chk-7f3a';
```

## B. How to make sure a payment is not repeated

- Send the gateway an idempotency key or a unique receipt reference per payment so a retried API call cannot create two charges.
- Store the gateway **payment id** with a **unique constraint**; a webhook for an id already processed is acknowledged (HTTP 200) and ignored.
- One order has at most one **successful** payment. If a second success arrives, auto-refund it and alert.
- **Refunds:** an idempotency key per refund, and the refunded total never exceeds the paid total.
- Reply quickly to a webhook and do heavy work asynchronously, so the gateway does not retry unnecessarily.

## C. How to make sure no customer is repeated

- Make the **verified mobile number** (and email, where used) **unique**, after normalising the format (+91, spaces, case).
- Verify with an **OTP** before creating the account; use IDs from trusted logins (Google, Apple) as extra unique keys.
- On signup, **check then insert inside one transaction**, backed by the unique index, so two simultaneous signups cannot both win.
- **Duplicate detection and merging:** match on phone, email, device and name or address similarity; merge accounts with a documented process that moves orders, wallet and loyalty points and keeps an audit trail.
- **Abuse and promos:** limit first-order offers per device, phone, payment instrument and address to stop repeat sign-ups.
- **Privacy:** keep only needed identifiers, protect them, and honour deletion requests under the DPDP Act.

| Where duplicates come from | Fix |
|---|---|
| Double tap or slow screen | Disable the button; idempotency key |
| Client retry after timeout | Same key returns the same order |
| Two servers handle one request | Unique database constraint; short lock |
| Webhook sent twice | Dedupe by payment id; state machine |
| Queue redelivers a message | Idempotent consumer; processed-message table |
| Same person signs up twice | Unique verified phone; merge process |

> **Test it:** include duplicate scenarios in QA: double tap, kill the server mid-payment, replay the webhook five times, send the same signup from two phones. The target metric is **zero duplicate**

**orders and zero duplicate charges**, tracked daily.

**Key takeaways**

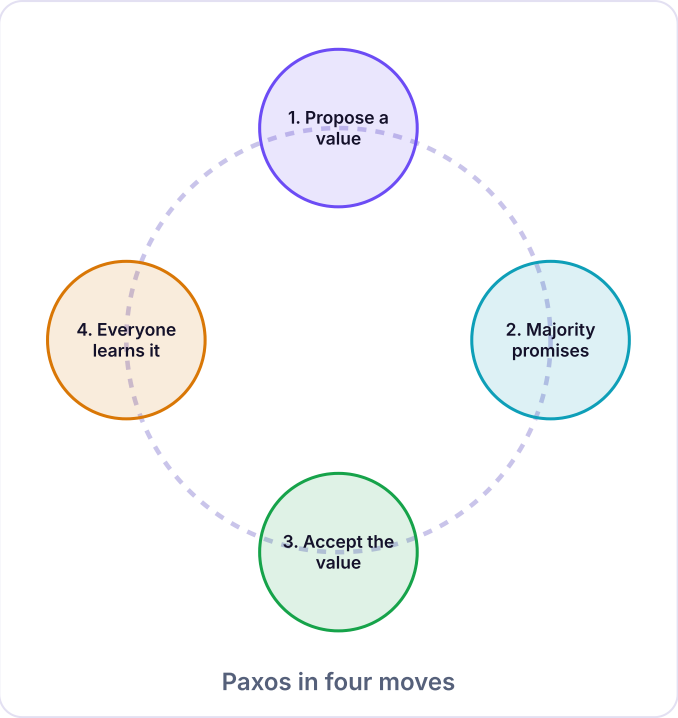
- Idempotency key plus a unique database constraint is the core defence.
- Make every webhook and queue consumer idempotent.
- Protect customers too: unique verified phone, OTP, merge process.

**Practice exercise:** Write the duplicate-protection checklist for your product's checkout, signup and refund flows.

ADVANCED · CLASS 13

# Consensus: Paxos and Raft without the maths

Understand why servers must agree and how they do it.




Paxos in four moves

If four servers each keep a copy of the truth ("order 55 is paid"), what if two disagree? **Consensus** is how a group of servers agrees on one value even when some crash or messages are delayed. It is the engine behind reliable leaders, locks and configuration.

- **Paxos** (Leslie Lamport): the classic algorithm. A *proposer* suggests a value, *acceptors* promise and accept, *learners* record the result. Two phases: **prepare/promise**, then **accept**. A value is chosen once a **majority (quorum)** accepts it.
- **Raft:** designed to be easier to understand. One **leader** takes all writes and copies them to followers. If the leader dies, a new election starts.
- **Majority math:** with N nodes you survive `floor((N-1)/2)` failures. 3 nodes tolerate 1, 5 nodes tolerate 2. The 4-node ring on the whiteboard tolerates only 1 (same as 3), which is why odd numbers are preferred.

| Where you meet it | What it protects |
|---|---|
| etcd (Raft), ZooKeeper (ZAB), Consul | Cluster settings, leader election, distributed locks |
| Google Spanner (Paxos) | Globally consistent database |
| Cassandra light-weight transactions | "Insert only if not exists" |
| Kafka (KRaft) | Keeping the message cluster's metadata consistent |

**What a PM should take away:** consensus gives *safety* but costs *speed*. Every write needs a majority round-trip, so spreading across regions makes writes slower. You rarely build it; you choose products that already use it, and you accept the latency and cost that come with strong consistency.

**Key takeaways**

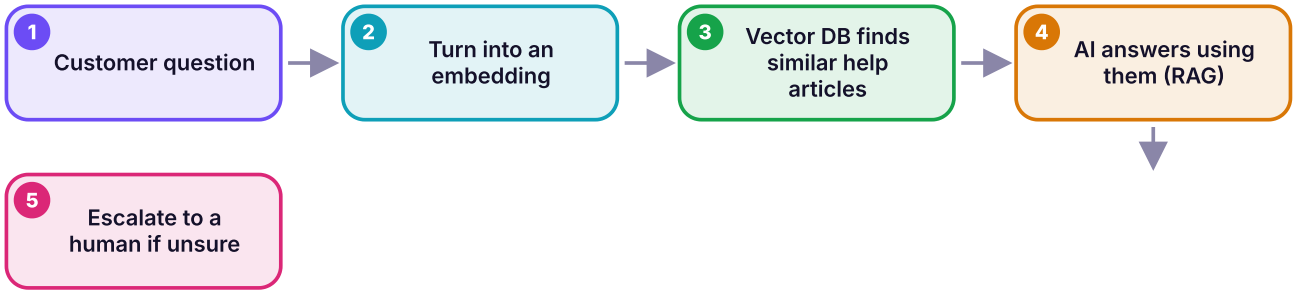
- Consensus = servers agreeing on one value despite failures.
- Majority (quorum) is the core idea; odd numbers of nodes are best.
- Strong agreement costs latency, a trade-off PMs must price in.

**Practice exercise:** Explain in three sentences to a friend why four partners must agree before a ledger entry is final.

ADVANCED · CLASS 14

# Vector databases, geolocation and the AI parts

Add meaning search, location and chatbots to the design.




A support chatbot with a vector database

## Vector databases

An **embedding** is a list of numbers that captures the *meaning* of text, an image or a product. Similar meanings sit close together. A **vector database** stores embeddings and finds the nearest ones quickly (**ANN**, approximate nearest neighbour, using methods like HNSW). So "something to have with chai" can find biscuits and rusks, even without those exact words.

| Use | Example |
|---|---|
| Semantic search | Search that understands intent and Hinglish |
| RAG chatbot | Find the right policy or help article, then let the AI answer from it |
| Recommendations | "People who like this also like..." |
| Duplicate and fraud detection | Near-identical listings or reviews |

- **Hybrid search:** combine keywords (BM25) with vectors, and add filters such as city, in stock, price.
- **Options:** a vector DB (Pinecone, Qdrant, Weaviate, Milvus) or the **pgvector** add-on to Postgres, often enough for starting out.
- **Watch:** re-embedding cost when content changes, relevance testing (evals), and privacy of what you embed.
- **Not for:** exact lookups like "order 55". Use SQL for those.

## Geolocation

- Store places as latitude and longitude; use **geohash** or Uber's **H3** hex grid to group nearby points fast.
- Tools: PostGIS (Postgres), Redis GEO, Elasticsearch geo queries, Google Maps APIs.
- Typical questions: which dark store is within 2 km and has stock? What is the ETA? Is this pin inside the service area?
- Privacy: ask consent, collect only what you need, follow the DPDP Act.

> AI and location features are additions to the same architecture: they read from the inventory and

orders data, they do not replace the SQL ledger.

**Key takeaways**

- Vector DBs search by meaning, not exact words.
- RAG uses a vector DB to ground chatbot answers.
- Geohash and H3 make 'nearest store' fast.

**Practice exercise:** Design the vector-search and location steps for 'find me a 10-minute dinner nearby'.

ADVANCED · CLASS 15

# The supporting cast: APIs, queues, events and observability

Know the other terms that appear in every design review.

| Term | Plain meaning | Why a PM cares |
|---|---|---|
| **API / REST / GraphQL** | The doors through which apps and partners talk to your service | Contract with partners; versioning breaks apps |
| **Monolith vs microservices** | One big app vs many small services | Speed of teams vs operational complexity |
| **Message queue (Kafka, RabbitMQ, SQS)** | A line where work waits to be processed later | Handles spikes, avoids blocking the user |
| **Event-driven / pub-sub** | Services announce "order created" and others react | Loose coupling, but harder to trace |
| **Webhook** | A partner calls your URL when something happens | Payment status updates; needs retries and idempotency |
| **Rate limiting / backoff** | Cap requests; wait longer between retries | Prevents abuse and retry storms |
| **Circuit breaker** | Stop calling a failing service for a while | One failure does not take everything down |
| **OLTP vs OLAP** | Live transactions vs analytics and reporting | Do not run heavy reports on the live database |
| **CDC** | Change data capture: stream database changes elsewhere | Feeds search, analytics and caches |
| **Feature flags / canary / blue-green** | Release to a few users or switch between two versions | Safe rollouts and quick rollback |
| **Observability** | Logs, metrics and traces to see what is happening | Faster incident response; ask for dashboards in specs |

## Back-of-the-envelope sizing (a PM skill)

1 million orders a day is about **12 orders per second** on average (1,000,000 / 86,400). Plan for peaks of 10x, so about **120 per second**. If each order takes 2 KB, that is about **2 GB a day** or roughly **730 GB a year**. Browsing is far larger than buying: expect 50 to 100 reads for every order, so cache hot data.

> Ask engineers: "What is our peak requests per second, our p99 latency, and what breaks first if traffic doubles?"

**Key takeaways**

- Know the vocabulary: API, queue, event, webhook, cache, observability.
- Queues and events absorb spikes but add complexity.
- Estimate scale with simple arithmetic before debating tools.

**Practice exercise:** Estimate requests per second and daily storage for an app with 200,000 orders a day.

ADVANCED · CLASS 16

# Case study and PM interview answer: design inventory and checkout

Practise a complete answer from requirements to trade-offs.

**Prompt:** "Design inventory and checkout for a 10-minute grocery delivery app."

1. **Clarify:** users, cities, stores, items per store, flash sales? Out of scope: delivery routing.
2. **Goals and metrics:** no overselling, checkout under 2 seconds p95, 99.95% availability, order success rate, refund rate.
3. **Scale:** 1M orders/day, peak 10x, 100 reads per order.
4. **High-level design:** app, API gateway, services (catalogue, inventory, order, payment, notifications), SQL for orders, key-value/Redis for carts and stock cache, Kafka for events, vector DB for support search, geolocation for the nearest store.
5. **Deep dive (inventory):** per-store stock in a fast store, atomic decrement, 8-minute holds, release on timeout, reconcile with the vendor app hourly.
6. **Failure modes:** payment succeeds but order write fails (saga plus idempotency), a node dies (replicas, quorum), vendor app offline (last-known stock, safety buffer).
7. **Trade-offs:** consistency for stock and money, availability for browsing, cost of real-time counts.
8. **Rollout:** pilot in one city, feature flags, dashboards, alerts, rollback plan.

| Interviewers reward | Interviewers penalise |
|---|---|
| Clarifying questions, metrics, trade-offs, user impact, simple numbers | Tool name-dropping, no numbers, ignoring failure cases, over-engineering |

**One-line answer:** "Strong consistency for stock and payments, eventual consistency for browsing, holds to protect the last item, idempotency to survive retries, and metrics to prove it works."

**Key takeaways**

- Answer in a fixed order: clarify, metrics, scale, design, deep dive, failures, trade-offs, rollout.
- Name the hard part and solve it.
- Tie every choice to a user or business outcome.

**Practice exercise:** Record yourself giving this answer in 5 minutes using the eight steps.

ADVANCED · CLASS 17

# Final project: write a one-page system brief

Produce a PM-friendly design brief you can share with engineers.

Choose a product (food delivery, ticket booking, wallet, ed-tech) and write a one-page brief that answers:

- What are the 5 core data types and which database fits each (SQL, key-value, document, wide-column, vector)?
- Which operations must be strongly consistent, and which can be eventually consistent?
- How do you prevent double selling or double charging?
- What scale and latency targets do you assume, and how did you estimate them?
- What are the top 3 failure scenarios and your response?
- What does it cost per 1,000 orders, and what is the lever to reduce it?

Share it with an engineer and ask for the three biggest risks they see. Update the brief with their answers.

**Key takeaways**

- A good brief is short, numeric and honest about trade-offs.
- Engineers improve a draft faster than a blank page.
- Keep the brief updated as the product scales.

**Practice exercise:** Complete the one-page system brief and review it with an engineer.

TOOLKIT

# Templates

## System design questions to ask engineers

| Field | Fill in |
|---|---|
| What is our peak requests per second and p99 latency? | |
| Which data must be strongly consistent? | |
| What happens if the payment succeeds but the order write fails? | |
| How do we avoid double selling and double charging? | |
| What breaks first at 2x traffic? | |
| What is the cost per 1,000 orders? | |
| What is the rollback plan? | |

## PM system design interview answer

| Field | Fill in |
|---|---|
| Clarify and scope | |
| Metrics and targets | |
| Scale estimate | |
| High-level design | |
| Deep dive on the hard part | |
| Failure modes | |
| Trade-offs | |
| Rollout and monitoring | |

## Technical trade-off record

| Field | Fill in |
|---|---|
| Decision | |
| Options considered | |
| Consistency need | |
| Cost and latency impact | |
| Risk | |
| Owner and review date | |

TOOLKIT

# Key terms cheat sheet

| Term / tool | Meaning / use |
|---|---|
| **SQL** | Tables with fixed structure, joins and strong guarantees |
| **NoSQL** | Flexible, scalable stores (key-value, document, wide-column, graph) |
| **ACID** | Atomicity, Consistency, Isolation, Durability: safe transactions |
| **BASE** | Basically Available, Soft state, Eventually consistent |
| **CAP** | In a network split choose consistency or availability |
| **Cassandra** | Masterless wide-column database for huge write volumes |
| **Paxos / Raft** | Algorithms that make servers agree on one value using a majority |
| **Quorum** | A majority of copies; R + W > N gives fresh reads |
| **Vector DB** | Stores embeddings and finds similar meaning fast |
| **Replication** | Copies of data on several servers |
| **Sharding** | Splitting data across servers by a key |
| **Cache** | Hot data kept in memory (Redis) |
| **Idempotency key** | Makes retries safe |
| **Saga** | A chain of steps with undo actions |
| **2PC** | Two-phase commit across systems |
| **Hold / reservation** | Temporary stock lock during checkout |
| **p95 / p99** | Latency of the slower 5% / 1% of requests |
| **SLA / SLO / SLI** | Promise / target / measurement |
| **Geohash / H3** | Grid systems for fast location lookup |
| **RAG** | AI answers grounded in retrieved documents |
| **LWT** | Cassandra lightweight transaction: IF NOT EXISTS / IF condition, slow because it uses Paxos |
| **Idempotent** | Repeating gives the same result as doing it once |
| **Stateless service** | Keeps no customer data in server memory, so copies can be added freely |
| **Autoscaling** | Adding or removing servers automatically with load |
| **Reconciliation** | Comparing your records with the gateway or bank and fixing mismatches |

# Glossary

| | |
|---|---|
| **API** | A defined way for software to talk to other software |
| **Microservice** | A small, independently deployed service |
| **Monolith** | One large application |
| **Load balancer** | Spreads traffic across servers |
| **CDN** | Network of servers serving files near users |
| **Index** | Lookup structure that speeds reads |
| **Primary / foreign key** | Unique row ID / link to another table |
| **Join** | Combine rows from tables |
| **Schema** | The structure of a database |
| **Transaction** | A group of steps that succeed or fail together |
| **Race condition** | A bug when simultaneous actions interfere |
| **Pessimistic / optimistic lock** | Lock first / check for change at commit |
| **Eventual consistency** | Copies agree after a short delay |
| **Hot partition** | One shard or node getting too much traffic |
| **Replication factor** | Number of copies |
| **LSM tree** | Write-friendly storage design used by Cassandra |
| **Embedding** | Numbers capturing meaning |
| **ANN / HNSW** | Fast approximate nearest-neighbour search |
| **Hybrid search** | Keywords plus vectors |
| **pgvector** | Vector search add-on for PostgreSQL |
| **Kafka** | A distributed event streaming platform |
| **Webhook** | Partner calls your URL on an event |
| **Circuit breaker** | Stops calls to a failing dependency |
| **Backoff** | Waiting longer between retries |
| **OLTP / OLAP** | Live transactions / analytics |
| **CDC** | Streaming database changes |
| **RPO / RTO** | Data-loss limit / downtime limit |
| **Observability** | Logs, metrics, traces |
| **DPDP Act** | India's Digital Personal Data Protection Act, 2023 |
| **PCI-DSS** | Security standard for card data |
| **Gossip / vnodes / SSTable / compaction / tombstone** | Cassandra internals: node chat, ring slices, disk files, merging, delete markers |
| **Horizontal / vertical scaling** | More machines / a bigger machine |

| | |
|---|---|
| **Backpressure** | Slowing or refusing work when overloaded |
| **State machine** | The allowed order of statuses |
| **Outbox pattern** | Save the event with the data, then publish it |
| **Payment gateway** | Service that connects your app to banks, UPI and cards |
| **Settlement report** | Gateway file listing the money actually received |

INTERVIEW PREP

# 20 interview questions with answers

**Q1. Why do PMs need to know SQL vs NoSQL?**
Because the choice affects consistency, cost, speed and features. PMs should know SQL is best for money and orders, NoSQL for scale and flexibility, and most products use both.

**Q2. Explain ACID to a non-technical person.**
It is four promises that make a transaction safe: all-or-nothing, rules always hold, simultaneous users do not interfere, and confirmed data survives a crash.

**Q3. What is the CAP theorem?**
In a network split a system must pick between consistency (same truth everywhere, may refuse requests) and availability (always answer, maybe stale). Pick consistency for payments, availability for browsing.

**Q4. What is Cassandra good at?**
Very high write volume, multi-region and always-on use cases like messages, feeds and event logs. It is weak at joins and strict transactions.

**Q5. What is Paxos and why does it matter?**
An algorithm that lets servers agree on one value using a majority, even with failures. It underpins reliable leaders, locks and strongly consistent databases, at the cost of some latency.

**Q6. Paxos vs Raft?**
Both achieve consensus. Raft is simpler to understand and implement, with a clear leader and elections; Paxos is the older, more general form.

**Q7. How do you prevent selling the last item twice?**
Make check and decrement one atomic step, use holds with timeouts, add idempotency keys, and reconcile stock with vendors.

**Q8. What is idempotency and why for payments?**
Repeating a request has the same effect as doing it once. It stops double charges when users retry or a payment callback arrives twice.

**Q9. When would you use a vector database?**
For meaning-based search, RAG chatbots, recommendations and near-duplicate detection. Not for exact lookups.

**Q10. What is RAG?**
Retrieval-augmented generation: fetch relevant documents (often via a vector DB) and let the AI answer using them, which reduces made-up answers.

**Q11. Replication vs sharding?**
Replication copies the same data for safety and read speed; sharding splits different data across servers for capacity.

**Q12. How would you estimate load for 1M orders a day?**
About 12 per second on average, 120 per second at a 10x peak, 2 GB of order data a day, and many more reads than writes.

**Q13. What would you ask engineers before a sale?**
Expected peak requests per second, p99 latency, what fails first, load-test results, rollback plan and

on-call coverage.

**Q14. Monolith or microservices?**
Start simple; split services when teams and scale require it. Microservices bring speed for teams but add network, data and debugging complexity.

**Q15. What is a Cassandra LWT?**
A lightweight transaction (IF NOT EXISTS or IF condition) that runs a Paxos round for a safe check-then-write. It is correct but slow, so use it only for rare, important actions like claiming a unique key.

**Q16. What happens when traffic grows 20x?**
Add a load balancer and many identical stateless servers with autoscaling, cache hot data, add read replicas, move slow work to queues, rate limit, and switch off non-essentials. The next bottleneck is usually the database or a third-party API.

**Q17. How do you stop duplicate orders?**
Use an idempotency key per checkout attempt, a unique database constraint on it, a short lock per cart, an order state machine and a disabled button. Replays return the same order.

**Q18. Why is payment confirmed by webhook?**
The user may close the app or the screen may be wrong. The gateway tells your server the verified result, so the server decides, and reconciliation fixes any mismatch.

**Q19. How do you avoid duplicate customers?**
A unique, normalised, OTP-verified phone number backed by a database constraint, a transactional check-then-insert, a documented merge process and promo limits.

**Q20. What is a saga?**
A series of local transactions with compensating undo steps (refund, release stock) used instead of one giant distributed transaction.

LEARN MORE

# Further reading and tools

- System Design Primer (free, GitHub): https://github.com/donnemartin/system-design-primer
- Designing Data-Intensive Applications (book site): https://dataintensive.net/
- ByteByteGo system design: https://bytebytego.com/
- Raft visualisation and paper: https://raft.github.io/
- Paxos Made Simple (Lamport): https://lamport.azurewebsites.net/pubs/paxos-simple.pdf
- Apache Cassandra: https://cassandra.apache.org/
- PostgreSQL documentation: https://www.postgresql.org/docs/
- Redis documentation: https://redis.io/docs/
- Apache Kafka documentation: https://kafka.apache.org/documentation/
- pgvector (vector search for Postgres): https://github.com/pgvector/pgvector
- Uber H3 (hex grid): https://h3geo.org/
- Google SRE book (free): https://sre.google/sre-book/table-of-contents/
- Microservices (Martin Fowler): https://martinfowler.com/articles/microservices.html

© SkillNavora · skillnavora.in · Part of the AI Product Management and GenAI Launchpad programs. Practise with the free AI Tutor, AI PM Toolkit and Practice Lab on skillnavora.in.